

IYO One QCC5181 Firmware Factory Tool Bringup and Testing Guide

last updated: Jan 13, 2025

who: name (+role)

a. George Chen (george@iyo.dev) - wrote document and test commands

b.

c.

Intro

This covers the IYO One QCC5181 firmware bringup and testing guide. **It covers work intended for Protobuild Only, and excludes work on secure flashing.**

Helpful user guide to work alongside this is *80-CH362-1 CSRA681xx, QCC512x-8x, and QCC302x-8x Production Line Setup User Guide*

Hardware Requirements

For Windows: Windows(R) 10 (64-bit) on x86_64 that can install BlueSuite Tools

Contents

Instructions

Setup

Install Qualcomm Tools

BlueSuite v4.0

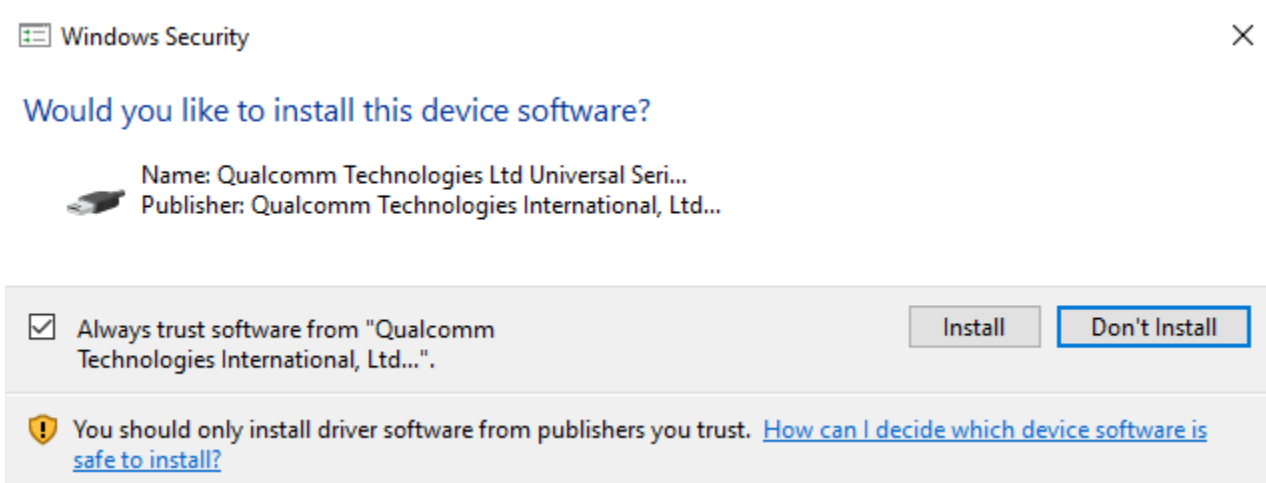
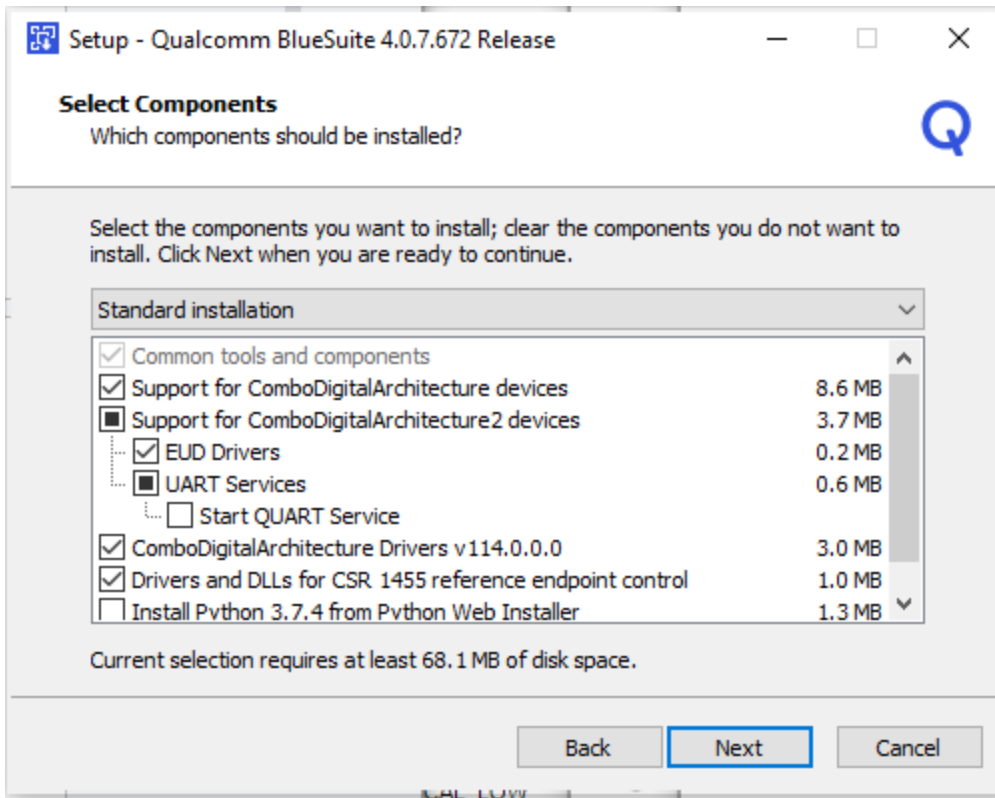
BlueSuite 4.0 contains tools + USB drivers needed to interface with the QCC5181. It can be installed from the Qualcomm webpage:

<https://www.qualcomm.com/support/software-tools/tools.qualcomm-bluesuite-development-suite.ac2f3465-793f-406d-ba4d-066a63b5a71f#overview>

Or from lyO Google Drive: [BlueSuite.WIN.4.0 Installer_4.0.7.672.zip](#)

It is necessary to install v4.0 because PSKey lengths increased to 1000 octets in the v4.0 release.

Go through the installation process, install Python 3.74 (if not already installed), and install the Qualcomm USB Hub Driver.



Summary of important tools/libraries

- ConfigCmd.exe provides a cli interface to change Configuration values stores as Persistent Storage Keys (PSKey). ConfigApp.exe is a GUI tool that uses ConfigCmd.exe to ease usage
- BlueTest3.exe provides a GUI interface for RF testing.
- NvsCmd.exe provides a cli interface to flash images, identify devices, and reset the device. NvsApp.exe provides is a GUI tool that uses NvsCmd.exe to ease usage.
- TestEngine.dll provides the TestEngine Interface

- DtsTerminal provides a python interface to the Bluetooth SPP Device Test Service.

Unlock Qualcomm USB Debug Transport using TransportUnlock.exe

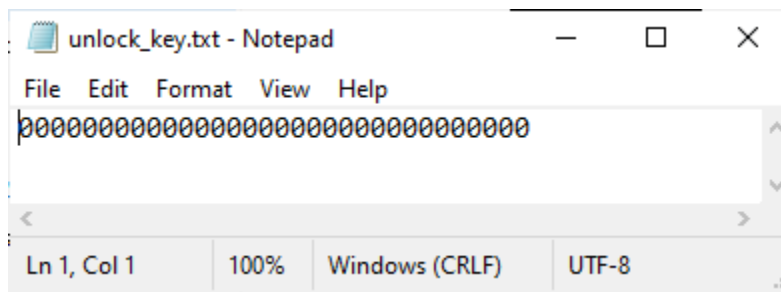
Unlocking the Qualcomm USB Transport is necessary to flash the QCC5181 over USB, and to access the USB Test Engine Service needed for board bringup testing, if a bluetooth antenna has not been installed. The protobuild of Iyo One QCC5181 firmware will not have the eFuse programmed with a key, therefore will have an unlocked transport - later builds may have the eFuse pre-programmed, and therefore will require an unlock key in order to connect over USB.

Refer to *80-CC094-1 Combo Digital Architecture Drivers User Guide* for more information and troubleshooting

The Qualcomm CDA driver and Qualcomm USB Debug drivers should be installed as part of installing BlueSuite v4.0. If not, they can be reinstalled again using Qualcomm_Drivers_WIN_CDA_xxx.exe in the drivers/ folder in the BlueSuite installation location

To enable USB Debug Transport Access, perform the following:

1. Create a text file containing the unlock code stored in the eFuse of the chip. An unprogrammed eFuse will have the code be 32 zeros. Refer to **trunlock.chm** in help folder of BlueSuite installation directory for more information Key Files. See also example: [unlock_key_unprogrammed.txt](#)



2. Connect the USB cable to the QCC5181 on Iyo One
3. Use Command Prompt to go to the BlueSuite installation directory, and type **TransportUnlock.exe writeunlockkey <unlock_key.txt location>**

```
Command Prompt
Microsoft Windows [Version 10.0.19045.5011]
(c) Microsoft Corporation. All rights reserved.

C:\Users\<redacted>>cd C:\Program Files (x86)\QTI\BlueSuite 4.0.7

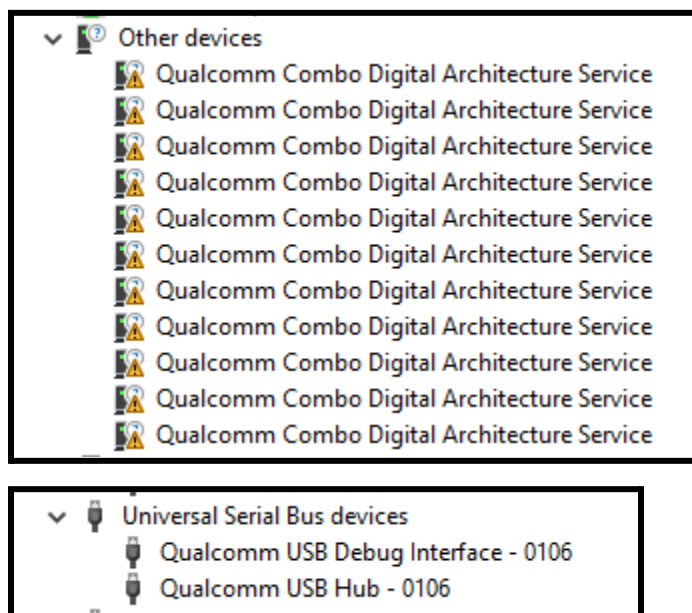
C:\Program Files (x86)\QTI\BlueSuite 4.0.7>TransportUnlock.exe writeunlockkey <redacted>\unlock_key.txt
TransportUnlock, version 4.0.7.672 Release
Copyright (c) 2010-2024 Qualcomm Technologies International, Ltd.
All Rights Reserved.
Qualcomm Technologies International, Ltd. Confidential and Proprietary.

<output key file> not given for "writeunlockkey" - defaulting to <redacted>\AppData\Roaming\QTI\KeyFile.txt'
Key file written for USBDBG
Registry setting written for USBDBG
Success

C:\Program Files (x86)\QTI\BlueSuite 4.0.7>
```

The unlock key should be added to the USBDBG registry.

4. Unplug and replug the USB cable connected to the QCC5181 on Iyo One. If successful, a Qualcomm USB Debug Interface should be present in Device Manager, and the Qualcomm Combo Device Architecture Service should be shown in Other devices



Board Bringup

Firmware Flash (NvsApp.exe, NvsCmd.exe)

A .xuv file should be provided by the Iyo Team intended as the golden production image.

For protobuild:

~~[20241031171005_iyo_one_factory_image_proto_flash_image.xuv](#)~~

For protobuild, that includes schemeB charger case communications:

~~[20241108161045_iyo_one_factory_image_proto_with_dfu_cc_schb_flash_image.xuv](#)~~

For protobuild, which include SW5100 boot on startup:

~~[20250113112736_iyo_one_factory_image_proto_with_cc_schb_sw5100_boot.xuv](#)~~

~~[20250221173009_iyo_one_factory_image_proto_with_cc_schb_1500K_sw5100_boot.xuv](#)~~

~~[20250309141921_iyo_one_factory_image_proto_with_cc_schb_1500K_sw5100_boot.xuv](#)~~

~~[20250312112154_iyo_one_factory_image_proto_with_cc_schb_1500K_sw5100_boot.xuv](#)~~

~~[20250330091150_flash_image_iyo_one_factory_evt_with_cc_schb_1500K_sw5100_boot.xuv](#)~~

~~[20250402185523_flash_image_iyo_one_factory_evt_with_cc_schb_1500K_sw5100_boot.xuv](#)~~

~~[20250412164247_flash_image_iyo_one_factory_evt_with_cc_schb_1500K_no_sw5100_boot.xuv](#)~~

~~[20250429103055_flash_image_iyo_one_factory_test_v1_2_0.xuv](#)~~

For protobuild, which does not boot SW5100 on startup:

~~[20250313115229_iyo_one_factory_image_proto_with_cc_schb_1500K_no_sw5100_boot.xuv](#)~~

~~[20250402185407_flash_image_iyo_one_factory_evt_with_cc_schb_1500K_no_sw5100_boot.xuv](#)~~

~~[20250412162953_flash_image_iyo_one_factory_evt_with_cc_schb_1500K_sw5100_boot.xuv](#)~~

~~[20250429110234_flash_image_iyo_one_factory_test_no5100boot_v1_2_0.xuv](#)~~

For EVT, to perform charger communication testing:

~~[20250429102012_flash_image_iyo_one_factory_charge_comms_test_v1_2_0.xuv](#)~~

For EVT, with SW5100 VUSB Enabled:

~~[20250429104145_flash_image_iyo_one_factory_test_enable_VUSB_v1_2_0.xuv](#)~~

NvsCmd.exe can be used to flash the image over command prompt - reference

NvsCmd.exe -help

Over command prompt for usage instructions

Example flash command over the USB debug interface (that is transport unlocked is:

```
NvsCmd.exe burn <flash_image.xuv> -deviceid 4 0 -usbdbg <port> -nvstype SQIF
```

Example flash command over the TRB interface is

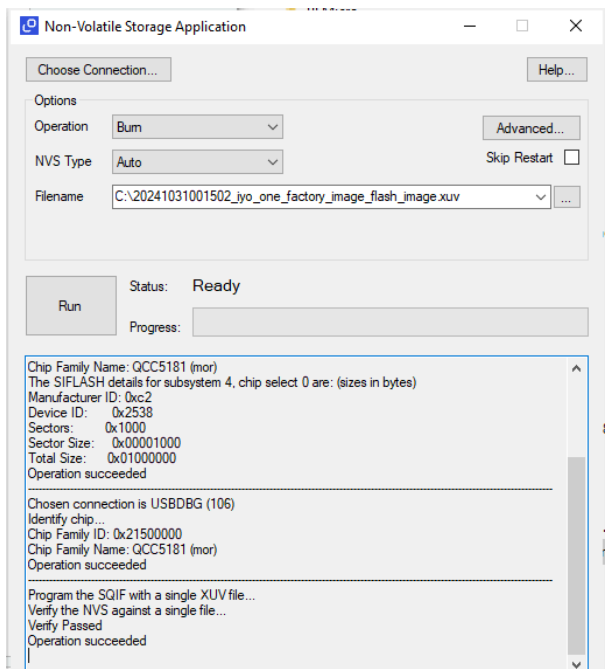
```
NvsCmd.exe burn <flash_image.xuv> -deviceid 4 0 -trb <port> -nvstype SQIF
```

```
C:\Program Files (x86)\QTI\BlueSuite 4.0.7>NvsCmd.exe burn [REDACTED]_iyo_one_factory_image_flash_image.xuv -deviceid 4 0 -usbdbg 106 -nvstype SQIF
nvscmd, version 4.0.7.672 Release
Copyright (c) 2010-2024 Qualcomm Technologies International, Ltd.
All Rights Reserved.
Qualcomm Technologies International, Ltd. Confidential and Proprietary.

Program the SQIF with a single XUV file...
Erasing sectors - this can take some time, with progress at 0%...
Programming...
Verify the NVS against a single file...
Verify Passed
100%
Success
```

Ensure that firmware verify passes, and flash is successful

NvsApp.exe can also be used, but is not intended for mass production use



For mass production use, Qualcomm recommends using the MultiProgCmd.exe application, or the TestFlash DLL API. Please use either tool to enable mass production factory setup.

Update Bluetooth Mac Address + Serial Number PsKey with Wired Interface (ConfigCmd.exe/ConfigApp.exe)

Each pair of units should have the same MAC address, except for the last bit, which indicates if the device is left or right on startup (1) or (0), e.g.

BD_ADDRESS = [10 F0 FF 5B 02 00] is the MAC address of a “right” device

BD_ADDRESS = [11 F0 FF 5B 02 00] is the MAC address of a “left” device

If wired access is available, ConfigCmd.exe can be used to change the bluetooth address, and other device configuration parameters. ConfigApp.exe is a GUI tool helpful for evaluating, but not ideal for mass production. *Note that if it is desired for Bluetooth MAC address + Serial Number to be updated after the device is fully enclosed, and functional tests pass, the Bluetooth SPP Device Test Service can also be used when the device is in production test mode.*

Using ConfigCmd.exe to change the serial number, mac address, and other device configuration parameters requires a text file like below. The command **txt2dev** can then be used to upload the configurations to the device. A QCC5181 .sdb database file is also needed to for the tool to work, which can be downloaded from the ADK, or from lyO google drive:

[qcc518x_qcc308x_hydracore_config.sdb](#)

See example file:

[dev_cfg_config_example.txt](#)


```
dev_cfg_config_example.txt - Notepad
File Edit Format View Help
version=QCC518X_CONFIG

#####
file=system3

# The capacitance for driving a crystal providing the raw clock
XtalLoadCapacitance = 0x0F

# Fine frequency trim for a crystal providing the raw chip clock
XtalFreqTrim = 0

#####
file=bt2

# BD_ADDRESS|
BD_ADDRESS = [ 10 F0 FF 5B 02 00 ]

#####
file=app3

# Device product string descriptor
USBProductString = "iyo one"

# Device serial number string descriptor
USBSerialNumberString = "012514512"

# Manufacturer string descriptor
USBManufString = "Iyo Inc"

#####
file=app5

# MIB key to set device name
DeviceName = "IYO ONE"
```

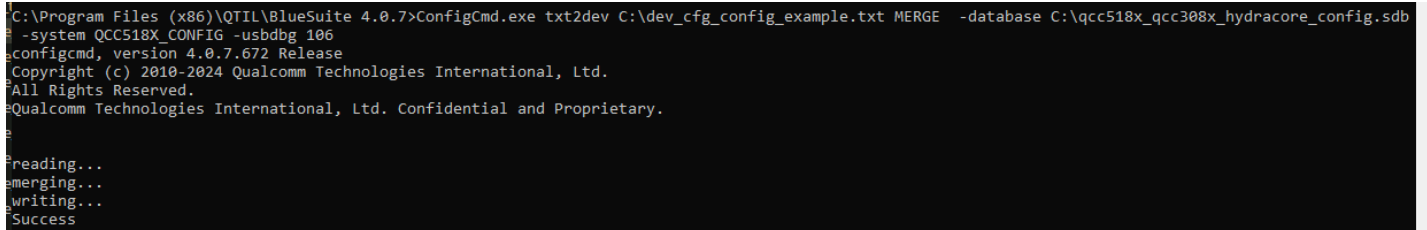
BD_ADDRESS (PSID=1024), and USBSerialNumberString (PSID=9) are the parameters in the .txt file to change to update the bluetooth mac address and the serial number of the device.

To update the dev_cfg with new serial number and MAC address over USB debug transport, the following command can be used:

```
ConfigCmd.exe txt2dev <txt file> MERGE -database <.sdb> -system  
QCC518X_CONFIG -usbdbg <port>
```

Using the TRB interface:

```
ConfigCmd.exe txt2dev <txt file> MERGE -database <.sdb> -system  
QCC518X_CONFIG -trb <port>
```



```
C:\Program Files (x86)\QTI\BlueSuite 4.0.7>ConfigCmd.exe txt2dev C:\dev_cfg_config_example.txt MERGE -database C:\qcc518x_qcc308x_hydracore_config.sdb  
-system QCC518X_CONFIG -usbdbg 106  
ConfigCmd, version 4.0.7.672 Release  
Copyright (c) 2010-2024 Qualcomm Technologies International, Ltd.  
All Rights Reserved.  
Qualcomm Technologies International, Ltd. Confidential and Proprietary.  
Reading...  
Merging...  
Writing...  
Success
```

The dev_cfg can be read out to confirm if the parameters were written successfully:

Using the USB debug transport:

```
ConfigCmd.exe dev2txt <txt file> -database <.sdb> -system QCC518X_CONFIG  
-usbdbg <port> -storeset dev_cfg
```

Using the TRB interface:

```
ConfigCmd.exe dev2txt <txt file> -database <.sdb> -system QCC518X_CONFIG  
-trb <port> -storeset dev_cfg
```

BT crystal calibration

Parameter XtalLoadCapacitance (PSID=1031) and XtalFreqTrim (PSID=1030) in the dev_cfg.txt file can be used to update the crystal capacitance and frequency trim to get the clock to adjust the clock properly for bluetooth radio. It additionally can be configured using the Test Engine API via API command *teConfigCacheWrite* and *teConfigCacheMerge*

```
dev_cfg_config_example.txt - Notepad
File Edit Format View Help
version=QCC518X_CONFIG

#####
file=system3

# The capacitance for driving a crystal providing the raw clock
XtalLoadCapacitance = 0x0F

# Fine frequency trim for a crystal providing the raw chip clock
XtalFreqTrim = 0
```

USBManufString PSID=7 (llyo Inc)
USBProductString PSID=8 (iyo one)

DeviceName PSID=315
USBManufString PSID=7 (llyo Inc)
USBProductString PSID=8 (iyo one)

Board Check using Qualcomm USB/TRB Test Engine

The Qualcomm Test Engine API is an interface for the development of production line test system for testing the QCC5181 over a wired interface. Refer to **TestEngine.chm** in the help folder of BlueSuite installation directory for more information on the API, and the [factory tools.zip](#) provides an example of how to use the API over a python interface.

Board Check commands

The *teAppWrite* and *teAppRead* TestEngine API command is used to issue production self-check tests and report results

Note that the commands sent follow the following 16-bit word order

0 (Test	1 (Test	2 (Device Test)
---------	---------	-----------------

Identifcation=PRODUCTIO N_TEST_ID)	Type=DEVICE _TEST)	
0x0001	0x0001	PCAL6408A = 0x0001 BHI360 = 0x0002 BHI360_SELF_TEST=0x0003 ADAU1787= 0x0004 MAX17055=0x0005 FIRMWARE_VERSION=0x0007 SHIPMODE=0x0008 PEER_PAIR_CHECK=0x0009 DELETE_HANDSETS=0x000A FACTORY_RESET=0x000B MAX17055_FORCE_CONFIG=0x000C MAX17055_FULL_DEVICE_ID=0x000D SW5100_POWERON_DEVICE_ID=0x000E SW5100_HARDRESET_DEVICE_ID=0x000F ADAU1787_DACOUT_HP0_ID = 0x0010 ADAU1787_DACOUT_HP1_ID = 0x0011 ADAU1787_DACOUT_HPBOTH_ID = 0x0012 ADAU1787_ADC01READ_ID=0x0013 ADAU1787_INNERMIC_LEVEL_ID=0x0014 SW5100_VUSB_DISABLE_LEVEL_ID=0x0015 SW5100_VUSB_ENABLE_LEVEL_ID=0x0016 BHI360_IMU_ACCEL_FOC_ID=0x0017 BHI360_IMU_GYRO_CRT_ID=0x0018 BHI360_IMU_GYRO_FOC_ID=0x0019 BHI360_IMU_ACCEL_CALIB_ID=0x001A BHI360_IMU_ACCEL_CALIB_STOP_ID=0x001B BHI360_IMU_GYRO_CALIB_ID=0x001C BHI360_IMU_GYRO_CALIB_STOP_ID=0x001D BHI360_IMU_MAG_CALIB_ID=0x001E BHI360_IMU_MAG_CALIB_STOP_ID=0x001F BHI360_IMU_ACCEL_READ_ID=0x0020 BHI360_IMU_GYRO_READ_ID=0x0021 BHI360_IMU_MAG_READ_ID=0x0022

		MICRECORD_SINGLE_START_ID = 0x8000 MICRECORD_SINGLE_48K_3000MS_MIC0_ST ART_ID = 0x8500 MICRECORD_STOP_ID = 0x9000 MICRECORD_READFLASH_START_ID=0xB00 0 MICRECORD_READFLASH_CONTINUE_ID = 0xB001
--	--	--

Response to commands follow the follow 16-bit word order:

0 (Test Identifcation=PR ODUCTION_TES T_ID)	1 (Test Type=DEVICE_T EST)	2 (Device Test)	3 (Pass, Fail)	4...onward
0x0001	0x0001	PCAL6408A = 0x0001 BHI360 = 0x0002 BHI360_SELF_T EST=0x0003 ADAU1787= 0x0004 MAX17055=0x0 005	Pass = 0x0001 Fail = 0x0000	Additional information for test result (if any)

Check Firmware Version

Issue command 0x0001, 0x0001, 0x0007 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)	4...onward
0x0001	0x0001	0x0007	Pass = 0x0001	Firmware version String.

The firmware version is ASCII string after byte 3 onwards. Format is:

Major.Minor.Patch.(buildName)

Perform PCAL6408A GPIO Expander Self-check

Issue command 0x0001, 0x0001, 0x0001 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0001	Pass = 0x0001

If the 3rd word is Fail =0x0000, then one of the PCAL6408A on the board could not be accessed. Expect BHI360, ADAU1787 to fail as well.

Perform BHI360 Device Self-check

This test checks if BHI360 was able to be booted successfully (it does not confirm if accelerometer, gyroscope, or magnetometer are functioning)

Issue command 0x0001, 0x0001, 0x0002 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
--	---------------------------	-----------------	----------------

CTION_TEST_ID)			
0x0001	0x0001	0x0002	Pass = 0x0001

If the 3rd word is Fail =0x0000, then the BHI360 IMU on the board could not be accessed.
Expect BHI360 Self test to fail as well.

Perform BHI360 Device IMU Self-test

This test perform the IMU Self-tests on the BHI360. The board should be kept still for this test
Issue command 0x0001, 0x0001, 0x0003 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 16 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0003	Pass = 0x0001
4 (accel test result)	5 (accel_x_offset)	6 (accel_y_offset)	7 (accel_z_offset)
BHY2_ST_PASSED = 0x00 BHY2_ST_X_AXIS_FAILED = 0x01 BHY2_ST_Y_AXIS_FAILED = 0x02 BHY2_ST_Z_AXIS_FAILED = 0x04 BHY2_ST_MULTI_AXIS_FAILURE = 0x07 BHY2_ST_NOT_SUPPORTED = 0x08 BHY2_ST_INVALID_PHYS_ID = 0x09 BHY2_ST_FAILED_PERFORM = 0xFFFF	any	any	any
8 (gyro test result)	9 (gyro_x_offset)	10 (gyro_y_offset)	11 (gyro_z_offset)
BHY2_ST_PASSED = 0x00 BHY2_ST_X_AXIS_FAILED = 0x01 BHY2_ST_Y_AXIS_FAILED = 0x02 BHY2_ST_Z_AXIS_FAILED = 0x04 BHY2_ST_MULTI_AXIS_FAILURE = 0x07 BHY2_ST_NOT_SUPPORTED = 0x08 BHY2_ST_INVALID_PHYS_ID = 0x09 BHY2_ST_FAILED_PERFORM = 0xFFFF	any	any	any

12 (mag test result)	13 (mag_x_offset)	14 (mag_y_offset)	15 (mag_z_offset)
BHY2_ST_PASSED = 0x00 BHY2_ST_X_AXIS_FAILED = 0x01 BHY2_ST_Y_AXIS_FAILED = 0x02 BHY2_ST_Z_AXIS_FAILED = 0x04 BHY2_ST_MULTI_AXIS_FAILURE = 0x07 BHY2_ST_NOT_SUPPORTED = 0x08 BHY2_ST_INVALID_PHYS_ID = 0x09 BHY2_ST_FAILED_PERFORM = 0xFFFF	any	any	any

If the 3rd word is Fail =0x0000, then the BHI360 IMU self-tests did not pass. Try again, or inspect results to identify which sensor is failing.

Perform ADAU1787 Device Self-check

This test checks if all ADAU1787's on the board could be accessed.

Issue command 0x0001, 0x0001, 0x0004 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 10 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0004	Pass = 0x0001
4 (adau1787 UAP result LSB)	5 (adau1787 UAP result MSB)	6 (adau1787 DMIC1 result LSB)	7 (adau1787 DMIC1 result MSB)
0 if no issues, otherwise error		0 if no issues, otherwise error	
8 (adau1787 DMIC2 result LSB)	9 (adau1787 DMIC2 result MSB)		
0 if no issues, otherwise error			

If the 3rd word is Fail=0x0000, then one of the ADAU1787's was unable to be accessed on the board. Inspect results to identify which one is not working.

Perform MAX17055 Device Self-check

This test checks if the MAX17055 could be accessed, and gets fuel gauge information.

Issue command 0x0001, 0x0001, 0x0005 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 20 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0005	Pass = 0x0001
4 (result LSB)	5 (result MSB)	6 (battery voltage mV)	7 (FOCV mV)
0 if no error, otherwise error		any	any
8 (avg current mA)	9 (soc)	10 (temp centiCelsius)	11 (full charge capacity mAH)
any	any	any	any
12 (remaining charge capacity mAH)	13 (time to empty min)	14 (time to full min)	15 (cycle count)
any	any	any	any
16 (nom avail capacity mAH)	17 (design voltage mV)	18 (desired voltage mV)	19 (desired charge current mA)
any	any	any	any

If the 3rd word is Fail =0x0000, then the MAX17055 did not pass. Inspect results.

Enter Shipmode

This informs the device to enter shipmode (power off, and turn off all peripherals, and only reboot from VCHG Signal) after 5V VCHG signal is removed.

Issue command 0x0001, 0x0001, 0x0008 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0008	Pass = 0x0001

Peer Pair Check

This checks if the device has successfully peer-paired

Issue command 0x0001, 0x0001, 0x0009 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Peer-paired success)
0x0001	0x0001	0x0009	Success = 0x0001

Delete Handsets

This deletes all previously paired handsets. The device will enable pair mode after all handsets have been deleted

Issue command 0x0001, 0x0001, 0x000A with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Delete handsets success)
0x0001	0x0001	0x000A	Success = 0x0001

Factory Reset

This performs a factory reset. The device will forget previously peer-paired device and try peer-pairing procedure again.

Issue command 0x0001, 0x0001, 0x000B with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x000B	Success = 0x0001

MAX17055 Force INI Configuration

This forces INI configuration setting on the MAX17055. INI configuration setting usually only is set after POR of the MAX17055 (when battery is first inserted).

Issue command 0x0001, 0x0001, 0x000C with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Force Write Success)
0x0001	0x0001	0x000C	Success = 0x0001

Perform MAX17055 Full Device Self-check

This test checks if the MAX17055 could be accessed, and gets full fuel gauge information.

Issue command 0x0001, 0x0001, 0x000D with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 35 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
---	----------------------------------	------------------------	-----------------------

0x0001	0x0001	0x0005	Pass = 0x0001
4 (result LSB)	5 (result MSB)	6 (battery voltage mV)	7 (FOCV mV)
0 if no error, otherwise error		any	any
8 (avg current mA)	9 (soc)	10 (temp centiCelsius)	11 (full charge capacity mAH)
any	any	any	any
12 (remaining charge capacity mAH)	13 (time to empty min)	14 (time to full min)	15 (cycle count)
any	any	any	any
16 (full avail capacity mAH)	17 (design voltage mV)	18 (desired voltage mV)	19 (desired charge current mA)
any	any	any	any
20 (timer LSB)	21 (timer MSB)	22 (current mA)	23 (nominal available capacity mAH)
any	any	any	any
24 (state of health %)	25 (VF SOC %)	26 (Avail SOC)	27 (Mix soc)
any	any	any	any
28 (available capacity mAH)	29 vf remaining capacity mAH	30 (Q residual mAH)	31 (Full Capacity mAH)
any	any	any	any
32 (RCell milliohms)	33 (Avg Temp centiC)	34 (Avg VCell mV)	
any	any	any	

If the 3rd word is Fail =0x0000, then the MAX17055 did not pass. Inspect results.

SW5100 Power On Command Reset

This toggles the SW5100 pin to begin power up.

Issue command 0x0001, 0x0001, 0x000E with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x000E	Success = 0x0001

SW5100 Hard Reset Command Reset

This holds the SW5100 power pin low for 20 seconds to induce a SW5100 hard reset

Issue command 0x0001, 0x0001, 0x000F with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x000F	Success = 0x0001

Set DAC Out to HP0 only

This sets the Audio Dac Out to HP0 only

Issue command 0x0001, 0x0001, 0x0010 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
--	---------------------------	-----------------	---------------------------

CTION_TEST_ID)			
0x0001	0x0001	0x0010	Success = 0x0001

Set DAC Out to HP1 only

This sets the Audio Dac Out to HP1 only

Issue command 0x0001, 0x0001, 0x0011 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x0011	Success = 0x0001

Set DAC Out to Both HP outputs

This sets the Audio Dac Out to both HP outputs

Issue command 0x0001, 0x0001, 0x0012 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x0012	Success = 0x0001

Perform ADAU1787 Device ADC01 Read

This test reads the ADC0/ADC1 of DMIC2 ADAU1787 for rotary encoder testing

Issue command 0x0001, 0x0001, 0x0013 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 10 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0013	Pass = 0x0001
4 (ADC0 LSB)	5 (ADC0 MSB)	6 (ADC1 LSB)	7 (ADC1 MSB)

If the 3rd word is Fail=0x0000, then the ADC0/ADC1 was unable to be accessed on the board. Inspect results to identify which one is not working.

Perform ADAU1787 Device Inner Mic level read

This test reads the innermic rms level of the UAP ADAU1787 for UAP detection testing.

Issue command 0x0001, 0x0001, 0x0014 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 10 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0014	Pass = 0x0001
4 (Inner mic level LSB)	5 (Inner mic level MSB)		

If the 3rd word is Fail=0x0000, then the ADC0/ADC1 was unable to be accessed on the board. Inspect results to identify which one is not working.

Set SW5100 VUSB GPIO to 0

This sets the SW5100 VUSB EN pin to low

Issue command 0x0001, 0x0001, 0x0015 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x0015	Success = 0x0001

Set SW5100 VUSB GPIO to 1

This sets the SW5100 VUSB EN pin to high

Issue command 0x0001, 0x0001, 0x0016 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x0016	Success = 0x0001

Perform IMU Accelerometer Fast Offset Compensation (FOC)

Performs the accelerometer fast offset compensation and stores offsets to NVM. The device should be held still.

Issue command 0x0001, 0x0001, 0x0017 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 6 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0017	Pass = 0x0001
4 (Foc Status)	5 (X_offset)	5 (Y_offset)	5 (Z_offset)
Pass= 0x00	Any	Any	Any

If the 3rd word is Fail=0x0000, then the FOC Failed.

Perform IMU Gyroscope Component Retrim (CRT)

Performs the gyroscope component retrim. The device should be held still

Issue command 0x0001, 0x0001, 0x0018 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x0018	Success = 0x0001

If the 3rd word is Fail=0x0000, then the CRT Failed.

Perform IMU Gyroscope Fast Offset Compensation (FOC)

Performs the gyroscope fast offset compensation and stores offsets to NVM. The device should be held still.

Issue command 0x0001, 0x0001, 0x0019 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 8 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0019	Pass = 0x0001
4 (Foc Status)	5 (X_offset)	6 (Y_offset)	7 (Z_offset)
Pass= 0x00	Any	Any	Any

If the 3rd word is Fail=0x0000, then the FOC Failed.

Perform IMU Accelerometer Calibration

Performs the accelerometer calibration and stores the calibration values to NVM. The device should be moved +Z/-Z, +Y/-Y, +X/-X to achieve an accuracy level of 3. Issue command 0x0001, 0x0001, 0x001A with *teAppWrite* , and then call *teAppRead* to read the change in accuracy state

Expected response back is 5 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x001A	Pass = 0x0001
4 (Accuracy Status)			
If accuracy is 0, then no axes are calibrated If accuracy is 1, then 1 axes are calibrated If accuracy is 2, then 2 axes are calibrated If accuracy is 3, then 3 axes are calibrated If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM			

If the 3rd word is Fail=0x0000, then the Calibration Failed and has stopped

If accuracy is 0, then no axes are calibrated

If accuracy is 1, then 1 axes are calibrated

If accuracy is 2, then 2 axes are calibrated

If accuracy is 3, then 3 axes are calibrated

If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM

Stop IMU Accelerometer Calibration

Stops an existing accelerometer calibration routine, if one is running.

Issue command 0x0001, 0x0001, 0x001B with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x001B	Success = 0x0001

Perform IMU Gyroscope Calibration

Performs the gyroscope calibration and stores the calibration values to NVM. The device should be held still to achieve an accuracy level of 3. Issue command 0x0001, 0x0001, 0x001C with *teAppWrite* , and then call *teAppRead* to read the change in accuracy state.

Expected response back is 5 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x001C	Pass = 0x0001
4 (Accuracy Status)			
If accuracy is 0, then no axes are calibrated If accuracy is 1, then 1 axes are calibrated If accuracy is 2, then 2 axes are calibrated If accuracy is 3, then 3 axes are calibrated			

If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM			
---	--	--	--

If the 3rd word is Fail=0x0000, then the Calibration Failed and has stopped

If accuracy is 0, then no axes are calibrated

If accuracy is 1, then 1 axes are calibrated

If accuracy is 2, then 2 axes are calibrated

If accuracy is 3, then 3 axes are calibrated

If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM

Stop IMU Gyroscope Calibration

Stops an existing gyroscope calibration routine, if one is running.

Issue command 0x0001, 0x0001, 0x001D with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x001D	Success = 0x0001

Perform IMU Magnetometer Calibration

Performs the magnetometer calibration and stores the calibration values to NVM. The device should be moved in a figure 8 motion to achieve an accuracy level of 3. Issue command 0x0001, 0x0001, 0x001E with *teAppWrite* , and then call *teAppRead* to read the change in accuracy state.

Expected response back is 5 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
---	----------------------------------	------------------------	-----------------------

0x0001	0x0001	0x001E	Pass = 0x0001
4 (Accuracy Status)			
If accuracy is 0, then no axes are calibrated If accuracy is 1, then 1 axes are calibrated If accuracy is 2, then 2 axes are calibrated If accuracy is 3, then 3 axes are calibrated If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM			

If the 3rd word is Fail=0x0000, then the Calibration Failed and has stopped

If accuracy is 0, then no axes are calibrated

If accuracy is 1, then 1 axes are calibrated

If accuracy is 2, then 2 axes are calibrated

If accuracy is 3, then 3 axes are calibrated

If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM

Stop IMU Magnetometer Calibration

Stops an existing magnetometer calibration routine, if one is running.

Issue command 0x0001, 0x0001, 0x001F with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Factory Reset Success)
0x0001	0x0001	0x001F	Success = 0x0001

Read IMU Accelerometer

Reads the accelerometer data (in digital counts)

Issue command 0x0001, 0x0001, 0x0020 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 7 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0020	Pass = 0x0001
4 (X)	5 (Y)	6 (Z)	
Any	Any	Any	

If the 3rd word is Fail=0x0000, then the read failed.

Read IMU Gyroscope

Reads the gyroscope data (in digital counts)

Issue command 0x0001, 0x0001, 0x0021 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 7 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0021	Pass = 0x0001
4 (X)	5 (Y)	6 (Z)	
Any	Any	Any	

If the 3rd word is Fail=0x0000, then the read failed.

Read IMU Magnetometer

Reads the magnetometer data (in digital counts)

Issue command 0x0001, 0x0001, 0x0022 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is 7 16-bit words, in the following order

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Pass, Fail)
0x0001	0x0001	0x0022	Pass = 0x0001
4 (X)	5 (Y)	6 (Z)	
Any	Any	Any	

If the 3rd word is Fail=0x0000, then the read failed.

Single Mic Recording Start

This performs a single mic recording. The Device Test 16-byte command is 0x8TSM, which each letter, T, S, M as 4-bit values:

Bits 0-3 M = Mic number	Bits 4-7 S = Sample Rate	Bits 8-11 T= time	Bits 12-15
0x0 = DMIC0 0x1 = DMIC1 0x2 = DMIC2 0x3 = DMIC3 0x4 = DMIC4 0x5 = DMIC5 0x6 = DMIC6 0x7 = DMIC7 0x8 = N/A 0x9 = N/A 0xA = FBANC Mic 0xB = FFANC Mic	0x0 = 48K 0x1 = 44.1K 0x2 = 24K 0x3 = 16K	0x0 = 500 msec 0x1 = 1000 msec 0x2 = 1500 msec 0x3 = 2000 msec 0x4 = 2500 msec 0x5 = 3000 msec 0x6 = 3500 msec 0x7 = 4000 msec 0x8 = 4500 msec 0x9 = 5000 msec 0xA = 5500 msec 0xB = 6000 msec 0xC = 6500 msec 0xD = 7000 msec 0xE = 7500 msec 0xF = 8000 msec	0x8

Issue command 0x0001, 0x0001, 0x8XXX with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identifcation=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (Mic recording status)
0x0001	0x0001	0x8TSM	Started= 0x0001 Memory full = 0x0002 Finished = 0x0003

Keep calling *teAppRead* to keep polling for the value.

Single Mic Recording Stop

This stops any mic recording. The Device Test 16-byte command is 0x9000

Issue command 0x0001, 0x0001, 0x9000 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back is

0 (Test Identifcation=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (success or fail)
0x0001	0x0001	0x9000	Success = 0x0001

Read Mic Recording from Flash Start

This prepares the system to read any stored mic recording from flash, and sets the read pointer to the beginning.

The Device Test 16-byte command is 0xB000

Issue commandS *teAppWrite* , and then call *teAppRead* to read result

Expected response back (if there is mic data, is)

0 (Test Identifcation=PRODU CTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (success or fail)
0x0001	0x0001	0xB000	Success = 0x0001
4 Mic Number	5 offset LSB	6 offset MSB	7 total length LSB
0-15			
8 total length MSB	9-1015 mic data		

Expected error response back (if there is no mic data, or error, is)

0 (Test Identifcation=PRODU CTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (success or fail)
0x0001	0x0001	0xB000	Fail = 0x0000

MICRECORD_READFLASH_START_ID=0xB000

MICRECORD_READFLASH_CONTINUE_ID = 0xB001

Read Mic Recording from Flash Continue

This reads the next block of stored mic recording from flash, after calling command 0xB000

The Device Test 16-byte command is 0xB001

Issue command 0x0001, 0x0001, 0xB001 with *teAppWrite* , and then call *teAppRead* to read result

Expected response back (if there is mic data, is)

0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (success or fail)
0x0001	0x0001	0xB001	Success = 0x0001
4 Mic Number	5 offset LSB	6 offset MSB	7 total length LSB
0-15			
8 total length MSB	9-1015 mic data		

Expected error response back (if there is no more mic data, or error, is)

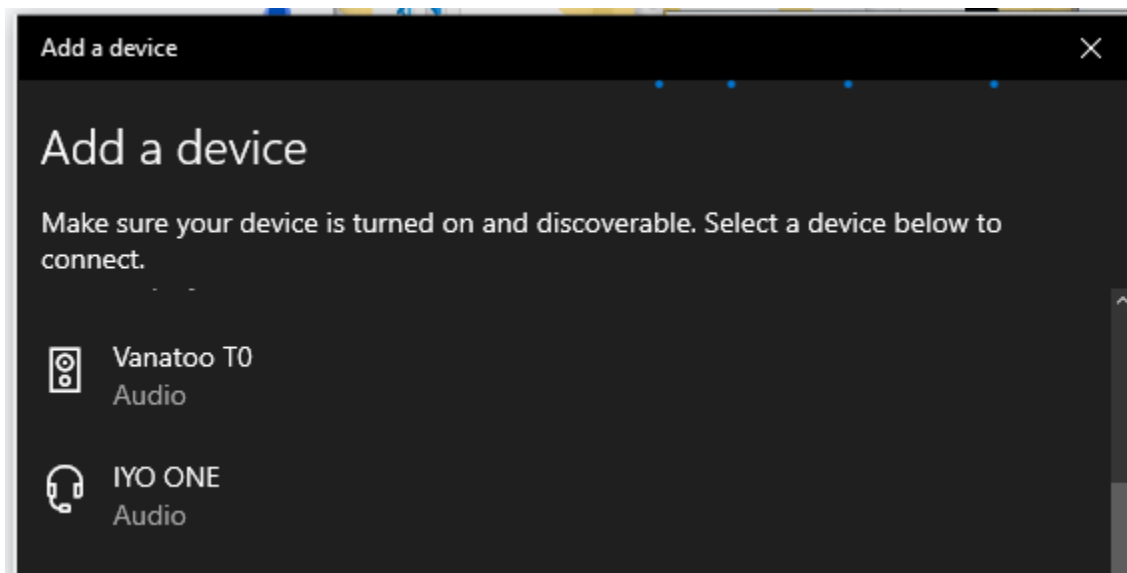
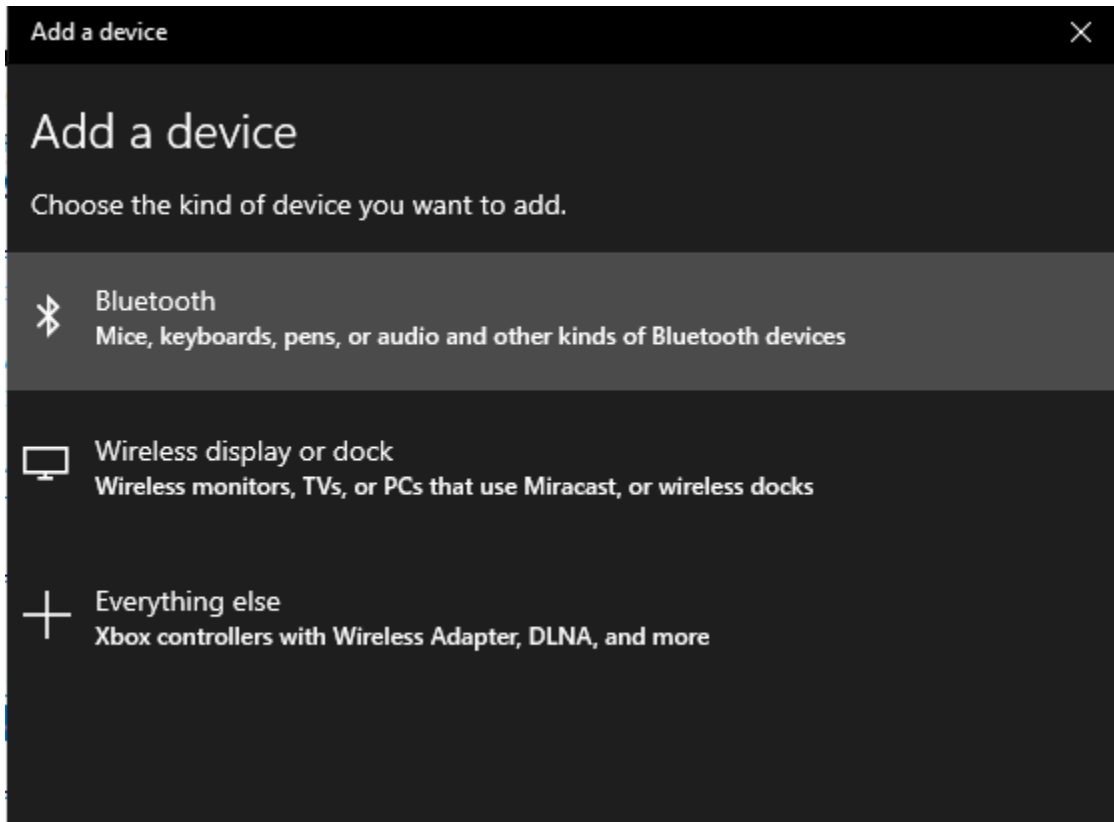
0 (Test Identification=PRODUCTION_TEST_ID)	1 (Test Type=DEVICE_TEST)	2 (Device Test)	3 (success or fail)
0x0001	0x0001	0xB001	Fail = 0x0000

Pre-Test/Functional Test

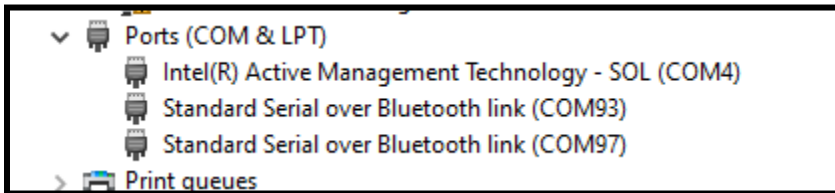
Pre-Tests and Functional Tests are performed over the Device Test Service Bluetooth SPP interface. Refer to *80-14839-1 Device Test Service User Guide* to instructions on standard QCC

Device Test Service AT Commands. IyO has added additional AT Commands for Factory Testing pertinent for IyO One hardware.

On a Windows machine, pair with the IyO One QCC5181 bluetooth device and connect



Once paired and connected, the IYO ONE QCC5181 device will show up as a Bluetooth Audio device (can stream audio over bluetooth for acoustic speaker tests), and a Standard Serial Bluetooth SPP COM Port.



Usage over python can be done using **DtsTerminal** folder in BlueSuite installation directory, and is also provided in [factory tools.zip](#). Note that factory tools.zip has changes to the python source to make the responses come faster.

To connect to the Device Test Service, connect to the Bluetooth COM Port and perform the Authentication challenge - **note that for the protobuild, the unlock key is the default**

See DtsTerminal/dts_terminal/dts_at_terminal.py for example on how to do that:

```
def authenticate_device(self):
    """authenticates connected device"""
    key = self.get_key()
    self.aes = AES.new(key, AES.MODE_CMAC)
    at_auth_start = ATCommand('AT+AUTHSTART' + AT_COMMAND_POSTFIX)
    response, success = self.transport.send_and_wait_response(at_auth_start)
    if success != "OK":
        return False
    random = response['+AUTHSTART']
    random = codecs.decode(random, 'hex')
    enc = self.aes.encrypt(random)
    enc_str = codecs.encode(enc, "hex")
    auth_resp = ATCommand('AT+AUTHRESP=' + enc_str.decode('utf-8').upper() +
AT_COMMAND_POSTFIX)
    _, success = self.transport.send_and_wait_response(auth_resp)
    if success != "OK":
        return False
    return True
```

Bluetooth SPP Device Test Service AT Commands

AT+TESTPIO

commands have been updated to also include access to the GPIO expanders

Pio's 0-63 reference the PIOs on the QCC5181, and PIO's 64-79 reference the GPIO expanders as such:

64 - PCAL6408A 0x20 pin 0
65 - PCAL6408A 0x20 pin 1
66 - PCAL6408A 0x20 pin 2
67 - PCAL6408A 0x20 pin 3
68 - PCAL6408A 0x20 pin 4
69 - PCAL6408A 0x20 pin 5
70 - PCAL6408A 0x20 pin 6
71 - PCAL6408A 0x20 pin 7
72 - PCAL6408A 0x21 pin 0
73 - PCAL6408A 0x21 pin 1
74 - PCAL6408A 0x21 pin 2
75 - PCAL6408A 0x21 pin 3
76 - PCAL6408A 0x21 pin 4
77 - PCAL6408A 0x21 pin 5
78 - PCAL6408A 0x21 pin 6
79 - PCAL6408A 0x21 pin 7

AT+TESTPIOOUTPUT=<pio>,<0,1>\r

Can be used to set the GPIO output to 0 or 1

AT+TESTPIOINPUTPIO=<pio>\r

Can be used to read the GPIO input

AT+MICRECORD

AT+MICRECORDSINGLE=<micNum>,<samplerate>,<durationMs>\r

Starts a microphone recording, given microphone number, sample rate, and length of time in milliseconds, and stores recording flash as 32-bit samples. Flash is first erased before the recording starts

micNum can be:

0 = DMIC0
1 = DMIC1
2 = DMIC2
3 = DMIC3
4 = DMIC4
5 = DMIC5
6 = DMIC6
7 = DMIC7
8 = N/A
9 = N/A
10 = FBANC Mic
11 = FFANC Mic

samplerate can be 48000, 44100, 32000, 24000, 16000, and 8000

returns

+MICRECORDSINGLE=0

when recording has started

+MICRECORDSINGLE=1

When recording has stopped writing to flash (still flushing data)

+MICRECORDSINGLE=2

When microphone recording has finished

The data can be read using **AT+MICRECORDREADFLASH**

AT+MICRECORDSINGLESTOP\r

Can stop a recording, if recording for too long

AT+MICRECORDREADFLASH\r

Reads off flash memory of the previous microphone recording

returns

+MICRECORDREADFLASH=<micNum>,<offset>,<num_bytes>,<total_bytes>

As header indicating the next set of lines of data

Next set of lines are 32 octets per line of data.

Example:

```
+MICRECORDREADFLASH:0,0,256,1024
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
+MICRECORDREADFLASH:0,256,256,1024
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
+MICRECORDREADFLASH:0,512,256,1024
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
+MICRECORDREADFLASH:0,768,256,1024
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
```

AT+MICRECORDREADFLASHCHUNK=<partition>,<chunkNumber>\r

Reads off a section flash memory of the previous microphone recording, given partition (0 for the first microphone recording, and chunk numbers

AT+MICRECORDREADFLASHCHUNK=0,0 for example, will read the first partition (first mic recorded), flash chunk 0 (the first 256 bytes)

returns

+MICRECORDREADFLASH=<micNum>,<offset>,<num_bytes>,<total_bytes>

As header indicating the next set of lines of data

Next set of lines are 32 octets per line of data.

Example:

+MICRECORDREADFLASH:0,0,256,1024

01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde
01234567890abcdef0123456789abcdef0234567890abcdef0123456789abcde

AT+IMU

AT+IMUACCELSSELFTEST\r

Performs the accelerometer self-test. The device should be held still

returns

+IMUACCELSSELFTEST:<result>,<x_offset>,<y_offset>,<z_offset>

If result is 0, then self-test passes

AT+IMUGYROSELFTEST\r

Performs the gyroscope self-test. The device should be held still

returns

+IMUGYROSELFTEST:<result>,<x_offset>,<y_offset>,<z_offset>

If result is 0, then self-test passes

AT+IMUMAGSELFTEST\r

Performs the magnetometer self-test. The device should be held still

returns

+IMUMAGSELFTEST:<result>,<x_offset>,<y_offset>,<z_offset>

If result is 0, then self-test passes

AT+IMUACCELFOC\r

Performs the accelerometer fast offset compensation and stores offsets to NVM. The device should be held still

returns

+IMUACCELFOC:<result>,<x_offset>,<y_offset>,<z_offset>

If result is 0, then fast offset compensation was successful

AT+IMUGYROCRTR\r

Performs the gyroscope component retrim. The device should be held still

returns

+IMUGYROCRTR:<result>

If result is 0, then the component retrim was successful

AT+IMUGYROFOC\r

Performs the gyroscope fast offset compensation and stores offsets to NVM. The device should be held still, and should be performed after a gyroscope component retrim

returns

+IMUGYROFOC:<result>,<x_offset>,<y_offset>,<z_offset>

If result is 0, then fast offset compensation was successful

AT+IMUACCELCALIB\r

Performs the accelerometer calibration and stores the calibration values to NVM. The device should be moved +Z/-Z, +Y/-Y, +X/-X to achieve an accuracy level of 3

returns

+IMUACCELCALIB:<accuracy>

If accuracy is 0, then no axes are calibrated

If accuracy is 1, then 1 axes are calibrated

If accuracy is 2, then 2 axes are calibrated

If accuracy is 3, then 3 axes are calibrated

If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM

AT+IMUACCELCALIBSTOP\r

Stops an existing accelerometer calibration routine, if one is running.

AT+IMUGYROCALIB\r

Performs the gyroscope calibration and stores the calibration values to NVM. The device should be held still to achieve an accuracy level of 3
returns

+IMUGYROCALIB:<accuracy>

If accuracy is 0, then no axes are calibrated

If accuracy is 1, then 1 axes are calibrated

If accuracy is 2, then 2 axes are calibrated

If accuracy is 3, then 3 axes are calibrated

If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM

AT+IMUGYROCALIBSTOP\r

Stops an existing gyroscope calibration routine, if one is running.

AT+IMUMAGCALIB\r

Performs the magnetometer calibration and stores the calibration values to NVM. The device should be moved in a figure 8 motion to achieve an accuracy level of 3
returns

+IMUACCELCALIB=<accuracy>

If accuracy is 0, then no axes are calibrated

If accuracy is 1, then 1 axes are calibrated

If accuracy is 2, then 2 axes are calibrated

If accuracy is 3, then 3 axes are calibrated

If accuracy is 4, then all axes are calibrated, and calibration is saved into NVM

AT+IMUMAGCALIBSTOP\r

Stops an existing magnetometer calibration routine, if one is running.

AT+IMUACCELREAD\r

Reads a sample of data off the accelerometer.
returns

+IMUACCELREAD:<x_data>,<y_data>,<z_data>

AT+IMUGYROREAD\r

Reads a sample of data off the gyroscope.
returns

+IMUGYROREAD:<x_data>,<y_data>,<z_data>

AT+IMUMAGREAD\r

Reads a sample of data off the magnetometer.

Returns

+IMUMAGREAD:<x_data>,<y_data>,<z_data>

AT+DEVICECHECK

AT+DEVICECHECK\r

Performs the device self-check, just like using Test Engine API

returns

+DEVICECHECK:<device>=<0|1>

<device> specifies the chip tested, and <0|1> is 0 if failed, 1 if passed

pcal6408a_0 = gpio expander 0x20 device address

pcal6408a_1 = gpio expander 0x21 device address

bhi360_0 = bhi360 IMU sensor

adau1787_0 = adau1787 UAP

adau1787_1 = adau1787 DMIC1

adau1787_2 = adau1787 DMIC2

max17055_0 = max17055 fuel gauge

Example:

+DEVICECHECK:pcal6408a_0=1

+DEVICECHECK:pcal6408a_1=1

+DEVICECHECK:bhi360_0=1

+DEVICECHECK:adau1787_0=1

+DEVICECHECK:adau1787_1=1

+DEVICECHECK:adau1787_2=1

+DEVICECHECK:max17055_0=1

OK

AT+FUELGAUGE

AT+FUELGAUGE\r

Reads off fuel gauge parameters.

Returns

+FUELGAUGE:<parameter>=<value>

For each parameter in the fuel gauge

Example:

```
+FUELGAUGE:voltage=4.328906V
+FUELGAUGE:vfocv=4.335234V
+FUELGAUGE:avg_current=-8mA
+FUELGAUGE:soc=99.644531
+FUELGAUGE:temp=27.175781C
+FUELGAUGE:full_charge_cap=342.500000mAH
+FUELGAUGE:rem_charge_cap=341.000000mAH
+FUELGAUGE:time_to_empty_min=2345.812000
+FUELGAUGE:time_to_full_min=0.000000
+FUELGAUGE:cycle_count=0
+FUELGAUGE:nom_avail_cap=295.000000mAH
+FUELGAUGE:design_volt=3.800000V
+FUELGAUGE:desired_volt=4.350000V
+FUELGAUGE:desired_charge_current=147.000000mA
OK
```

AT+ADAU1787

AT+ADAU1787ADC01READ\r

reads off the ADC0/ADC1 level values on the DMIC2 ADAU1787 for rotary encoder testing

Returns

+ADAU1787ADC01READ:<adc0>,<adc1>

<adc0> is the value read off of adc0

<adc1> is the value read off of adc1

Example:

```
+ADAU1787ADC01READ:6316,268430438
OK
```

AT+ADAU1787SETDACOUT=<dacNum>\r

Sets which dac should receive the output on the adau1787 for speaker output

<dacNum> can be:

0 = HP0

1 = HP1

2 = Send to HP0 and HP1

AT+SW5100

AT+SW5100POWERON\r

Toggles the SW5100 GPIO power pin (high (500ms), low (3s), high) to power on the SW5100.

Returns **OK** if success, **ERROR** if failed to toggle GPIO

AT+SW5100HARDRESET\r

Toggles the SW5100 GPIO power pin (high (500ms), low (15s), high) to cause a hard reset on the SW5100

Returns **OK** if success, **ERROR** if failed to toggle GPIO

Device Check

-Send AT Command AT+DEVICECHECK to perform self-check.

-Send AT Command AT+FUELGAUGE to check fuel gauge

IMU Calibration

-Send AT Command AT+IMUACCELSelfTEST to perform accelerometer self-test

-Send AT Command AT+IMUGYROSelfTEST to perform gyroscope self-test

-Send AT Command AT+IMUMAGSelfTEST to perform magnetometer self-test

-Send AT Command AT+IMUACCELFOC to perform accelerometer fast offset compensation

-Send AT Command AT+IMUGYROCRt to perform gyroscope component retrim

-Send AT Command AT+IMUGYROFOC to perform gyroscope fast offset compensation

-Send AT Command AT+IMUACCELCALIB to perform accelerometer calibration. Get to accuracy level of 3, and then 4 to confirm calibration stored in NVM

-Send AT Command AT+IMUGYROCALIB to perform gyroscope calibration. Get to accuracy level of 3, and then 4 to confirm calibration stored in NVM

-Send AT Command AT+IMUMAGCALIB to perform magnetometer calibration. Get to accuracy level of 3, and then 4 to confirm calibration stored in NVM

Microphone Recording/Testing

- Send AT+MICRECORDSINGLE=0,48000,1000 to record DMIC0 for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of DMIC0
- Send AT+MICRECORDSINGLE=1,48000,1000 to record DMIC1 for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of DMIC1\
- Send AT+MICRECORDSINGLE=2,48000,1000 to record DMIC2 for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of DMIC2
- Send AT+MICRECORDSINGLE=3,48000,1000 to record DMIC3 for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of DMIC3
- Send AT+MICRECORDSINGLE=4,48000,1000 to record DMIC4 for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of DMIC4
- Send AT+MICRECORDSINGLE=5,48000,1000 to record DMIC5 for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of DMIC5
- Send AT+MICRECORDSINGLE=6,48000,1000 to record DMIC6 for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of DMIC6
- Send AT+MICRECORDSINGLE=7,48000,1000 to record DMIC7 for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of DMIC7

- Send AT+MICRECORDSINGLE=10,48000,1000 to record FBANC Mic for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of FBANC Mic
- Send AT+MICRECORDSINGLE=11,48000,1000 to record FFANC Mic for 1 second (or longer)
- Send AT+MICRECORDREADFLASH after **+MICRECORDSINGLE=2** to read results of FFANC Mic

Speaker HP1/HP0 Testing

- Send AT+ADAU1787SETDACOUT=0 to test output on HP0. Stream audio over Bluetooth A2DP
- Send AT+ADAU1787SETDACOUT=1 to test output on HP1. Stream audio over Bluetooth A2DP
- Send AT+ADAU1787SETDACOUT=2 to test output on HP0 and HP1. Stream audio over Bluetooth A2DP

ADC_IN_0/ADC_IN_1 Testing

- Send AT+ADAU1787ADC01READ to read values on ADC0 and ADC1 pins

Set USR3 Key to 00 to remove production test mode.

To end production test mode (QCC5181 will perform peer-pairing to pair to another earbud to enable TWS), Send Device Test Service Command:

AT+DTSSETTESTMODE=0

And then AT+DTSENDTESTING=1 to end dts testing and reboot.

Alternatively, ConfigCmd.exe can also be used to change USR3 to 00 00, or TestEngine api can set it using teConfigCacheWriteItem

USR3 PSID=653

For more information, follow [IYO ONE QCC5181 Firmware Factory Peer Pair Guide](#)

Peer Pair with other earbud

Place IYO set close to each other after device reset and production test mode removed to begin the peer-pair process. The devices will be discoverable and pairable after peer-pairing is complete.

Perform Bluetooth DFU procedure

Use GAIA Client DFU service to upgrade the earbuds to intended customer image

~~[iyo_one_protobuild_QCC518X_dfu_file.bin](#)~~

~~[20241108160110_iyo_one_protobuild_with_cc_schemeB_QCC518X_dfu_file.bin](#)~~

[20250227181530_dfu_iyo_one_production_image_no_sw5100_boot.bin](#)

Verify Success+

GAIA Client should indicate the firmware version and build have changed to the appropriate version.

Future Work:

- Add steps for writing in secure keys (requires a Secure Lab)
- Encrypt image and load encrypted image
- Change Device Test Service Authentication Challenge Key
- Provide steps to lock USB debug transport, and remove USB connection at end production step.
- Change DFU signing key + encrypt DFU image
- Update USB Product ID/USB Vendor ID if there is decision to expose USB for customer use.